

REX MACHINA: THE CANON INDEX

Assignment #4, Dynamic Content Pipeline. Sritej Canchi.

A retrieval-grounded content pipeline for my capstone, *Rex Machina*: a shelter dog walks home to the family that gave it away and has to outwit the robot dog they replaced it with. Unreal Engine 5, Claude Haiku 4.5, solo build.

The knowledge base is my GDD. Not a summary of it, not lore written for this assignment.

`data/kb/gdd_rex_machina.md` is `my-capstone.md` byte for byte, the same source of truth Assignments #1 through #3 were built from. The pipeline chunks it into 39 pieces, indexes them as TF-IDF vectors, and retrieves against that index before any agent writes a word.

Output is three Unreal DataTables, 33 rows, each row carrying the provenance of how it got there.

```
python pipeline.py          # 33 rows, offline, about 2 seconds, no key needed
python test_pipeline.py     # 168 assertions
python evaluate_retrieval.py # the retrieval numbers below, recomputed
python estimate_cost.py     # the cost figure below, recomputed
python pipeline.py --live   # same run against claude-haiku-4-5
```

Standard library only. Nothing to install. No network. Section 12 has the live runbook.

1. The three gaps

The assignment asks for content the game specifically needs. Here is what was missing, with the line in the GDD that proves it was missing.

DT_NemesisReads, 24 rows

The fight surfaces 8 to 10 read lines per playthrough and the GDD contains one.

The Nemesis is the One Wow. Every round it predicts the dog's next tile, moves to cut it off, and prints a one-line read of what it noticed. GDD §4 gives exactly one of those lines, "Target favored left breaks four times in twenty.", and one category, `sealing_left`. The same section requires the UI to suppress any line whose `read_category` matches the last one shown, which is a rule that needs a category set to operate on. GDD §6.5 promises graceful degradation and specifies a chase-step default for the move, and says nothing about the line, so a fight with the API down is currently a silent one.

24 rows: 8 read categories across 3 charge bands. The categories come from the input contract in GDD §4, so each one is a read the machine can actually take: `sealing_direction`, `cover_habit`, `exit_fixation`, `periodicity_called`, `stall_detected`, `gait_read`, `charge_strain`, `prior_attempt`.

The charge bands come from one sentence in GDD §3: "Read register shifts with charge: clinical >60%, confident 30-60%, strained <30%." The GDD names the three registers and never says what they sound like. The pipeline settles that, and I want to be clear it is a design decision rather than something retrieved: clinical is telemetry with no second person and no

contractions, confident addresses the dog directly, strained is five words or fewer. Rule N6 enforces it.

DT_JourneyHazards, 6 rows

Acts 1 and 2 span nine days and the GDD names one hazard.

GDD §4 ships a single Gauntlet output, `hazard: "alley_chase"`. GDD §2 says Act 1 is the tutorial for Act 3, which means every hazard owes the player one piece of the Act 3 grammar, and the GDD never says which hazard teaches what. There is also an unsatisfiable constraint sitting in the beat table: `limp_onset` fires when the condition flips to limping "after the third hazard", and there is only one hazard on the books.

Six hazards, day-stamped against the journey ledger: `shelter_gate` (0), `alley_chase` (1), `road_crossing` (2), `train_leap` (3), `diner_yard` (6), `fence_line` (7). Three of them damage. In day order the third damaging hazard is `fence_line` on day 7, which is where the GDD dates `limp_onset`. Rule H5 checks that at table level, so the constraint is now satisfiable and machine-verified.

`fence_line` is the row I am most pleased with. It puts the injury that starts the limp on a gap under a chain fence, which is the same geometry as the phase 2 boss arena, which is why the Nemesis has a `prior_attempt` read that says "you tried the fence". Three separate things in the GDD that had nothing to do with each other now rhyme.

DT_ArenaPhases, 3 rows

The only level I am building first is described in six words.

GDD §3 gives the three phases as "yard, fence gap, train yard" and says the arena "re-lays exits and cover" at 33% and 66% without saying where. GDD §6.1 requires at least two escape routes per phase, which is a claim about a layout that did not exist. GDD §8 has Slice 1 building this arena before anything else in the game.

Three rows: grid span, exit tiles, cover tiles, light, sound bed, props, what the gate opening looks like, and the journey beat each phase calls back to. Phase 2 uses the exact exits `[[0,6],[9,2]]` and cover `[[3,5],[5,7]]` from the worked example in the GDD's Nemesis input contract, so that example is now a real board position instead of illustrative numbers.

Each phase calls back to a beat the player has already lived: the yard to `the_street`, the fence gap to `limp_onset`, the train yard to `train_leap`. The last arena has an open boxcar door in it, which is the shape that saved the dog on day 3, and this time the machine is standing in front of it.

Against the Class 5 menu

Class 5 framed the choice as a list: dialogue lines, quest descriptions, item descriptions, lore entries, NPC backstories. Rex Machina has no merchants, no quest log and no NPC roster, so the list does not transfer item for item. The mapping is that read lines are dialogue, hazards are quest descriptions, and arena phases are lore entries. What does transfer is the forcing question, which asks which type is the biggest gap between the prototype and a full game. The three headers above answer that with counts rather than adjectives.

2. What the retriever actually is

Lexical, not neural. TF-IDF over unigrams and adjacent bigrams, sublinear term frequency times smoothed inverse document frequency, L2 normalised, ranked by cosine similarity. 39 chunks, 2,471 terms, pure standard library, rebuilt from source in well under a second.

I want to be straight about that choice rather than let "vector index" do work it has not earned. The corpus is one 12 KB document and its discriminating vocabulary is proper nouns and identifiers: `dir_freq_20` , `read_category` , `fence_gap` , `boxcar` , `periodicity` . Exact term overlap is the signal. A sentence encoder would add a 90 MB download and a dependency to a repository whose largest file is 12 KB, and it would blur the identifiers the queries key on. `Index.embed` is the seam. Swap that one method for an encoder and nothing downstream changes.

Chunking is structure-aware, and two decisions there did more for retrieval quality than anything else. Fenced code blocks are kept whole, because the JSON agent contracts are the highest-value chunks in the corpus and splitting one across a boundary destroys it. Markdown tables are split into one chunk per row with the header prepended, because the Chronicler beat table is eight independent facts wearing a table costume; indexed as a single blob, every beat query returned all eight.

The model is asked. The gate enforces.

Assignment #3 shipped this asymmetry and buried it. The Beat Foundry's Lorekeeper was an agent told to filter the bible by day stamp, but the Chronicler received that agent's prose rather than a Python-filtered packet, so the day rule actually bit at the gate, in rule R4, not at the prompt. The model was asked. The engine enforced. A3's README listed that as known limit 3.

Retrieval changes where the asymmetry sits rather than removing it. There is no Lorekeeper in this pipeline. Selection is `Index.search` , which is deterministic code with a logged cosine score per chunk, so the packet that reaches the generator is now enforced at the point of selection instead of being requested from a model and checked afterwards. That is the upgrade A3 promised, and it narrows what the gate has to defend against: the generator can no longer be handed a fact that should have been filtered out, because no model chose the facts.

It does not retire the gate, and §6 is the proof. The critic invented rule A5 against `limp_onset` , a beat id that is genuinely canon, and the repair stage obeyed it and wrote `shelter_escape` , which is not. Deterministic selection upstream did nothing to stop a model inventing a violation downstream. Every stage where a model still has an opinion still needs code behind it.

3. RAG evidence: query, retrieved chunk, output

Full traces for all three types are in `out/retrieval_traces.md` , regenerated on every run. One worked example here.

Query (`journey_hazard` , seed stage, for the `fence_line` row):

```
fence line day 7 act 2 a gap under a chain fence; this is the hazard that
costs the back leg and the geometry the phase 2 arena reuses
```

Top retrieved chunk, `gdd_rex_machina#022` , cosine 0.2213, kind `table_row` :

```
| Beat ID | Fires when | Words | Example line |  
| `limp_onset` | `condition` flips to "limping" after the third hazard | 20 |  
"The back leg has opinions now. You tell it the same thing you told the gate:  
not yet." |
```

Output row (abridged; full row in `out/DT_JourneyHazards.json`):

```
{  
  "HazardID": "fence_line", "Day": 7, "Act": 2,  
  "PursuerType": "two dogs on the far side, working the wire in step with one another",  
  "Telegraph": "pacing the rail", "WindowTurns": 3,  
  "FailCondition": "the wire takes the back leg; the limp starts here and never clears",  
  "CausesCondition": true, "Teaches": "pattern breaking",  
  "PlayerSees": "Two dogs pacing one wire, in step. In step means a beat when neither is at the  
gap.",  
  "Provenance": "generated"  
}
```

The retrieved chunk is doing visible work. `CausesCondition: true` and the day stamp exist because the chunk says the limp flips after the third hazard. "the wire takes the back leg" is the retrieved beat's own noun. The row did not come from a model's idea of what a fence is.

4. The retrieval tweak, measured

The rubric asks for one concrete tweak made to improve game-fit. There were two, and both were measured rather than eyeballed. Gold chunk sets are hand-labelled: I read all 39 chunks and marked the ones a human writing that content type would need. Numbers reproduce with `python evaluate_retrieval.py`.

CONTENT TYPE	QUERY	P@6	R@6	MAP
arena_phase	naive	0.75	0.50	0.69
arena_phase	tuned	0.67	0.67	0.57
arena_phase	union (shipped)	0.57	0.67	0.53
journey_hazard	naive	0.17	0.14	0.17
journey_hazard	tuned	0.50	0.43	0.50
journey_hazard	union (shipped)	0.70	1.00	1.00
nemesis_read	naive	0.67	0.57	0.67
nemesis_read	tuned	1.00	0.86	1.00
nemesis_read	union (shipped)	0.86	0.86	0.86

Mean across the three: naive P@6 0.53 / R@6 0.40, tuned 0.72 / 0.65, union 0.71 / 0.84.

Tweak one: field-expanded queries. My first queries were the obvious ones, "Nemesis read lines" and "journey hazards". `journey hazards` scored P@6 0.17. It returned the token budget table, the Chronicler contract, and one chunk worth having. Rewriting the query to name the schema fields and the canon identifiers that appear in the document (`window_turns`, `telegraph`, `boxcar`, `limp onset`, "telegraphs but never adapts") took it to 0.50. Mean recall went from 0.40 to 0.65.

Tweak two: a second retrieval stage. Even tuned, the hazard query pulled the mechanics and missed the beat rows. `limp_onset` is the chunk that dates the third damaging hazard to day 7, and it was not being retrieved at k=6, which means the generator was being asked to satisfy a constraint it could not see. The fix is a short seed query built from each row's own key, unioned with the shared query. Hazard recall went from 0.43 to 1.00 and MAP from 0.50 to 1.00.

Two honest notes on that table. `arena_phase` and `nemesis_read` lost a little precision to the union, because seed queries pull in chunks the shared query would not have. I kept it, because recall is what matters when the retrieved set is the generator's only source of truth, and the cap of 10 chunks keeps the prompt small either way. Second, `arena_phase` naive beats tuned on P@6. The naive query is two words and gets a tight, small result set; the tuned query trades some of that for the two chunks that carry the escape-route rule and the tile coordinates. I would rather have those.

One tweak I tried and rejected: adding "approach meter stamina win loss slice 1 greybox NavMesh" to the arena query. It swapped which gold chunks came back at identical P@6 and R@6. It recovered `#007` and `#042`, and it lost `#034`, the two-escape-routes rule, and `#011`, the tile coordinates. The two it lost matter more for writing an arena than the two it gained, so the query stayed as it was.

5. What the critic caught

Nothing in this section was staged. The generator, critic and repair replies in `transcript/` were produced by separate agents, each handed one prompt file and nothing else. None of them could see `validators.py`, `specs.py`, the fallback table or each other. The mistakes below are mistakes those agents actually made.

The critic raised **21 findings** across 33 rows. Four, before and after.

N2, the read-line oracle. The Assignment #2 stress test closed this exploit once already. It walked straight back in through the generator.

```
before  "Left four in twenty. Sealed."
critic  N2: "Sealed." is a forward-looking declaration of the intercept the
        machine is about to make, not a diagnosis of what it saw
after   "Left four in twenty."
```

N4, a claim the four-move window cannot support. The Nemesis remembers four moves. A read asserting a five-round history is asserting memory it does not have.

```
before  "You ended three of five rounds on cover."
critic  N4: a three-of-five cover history exceeds the 4-move sliding window,
        and no engine scalar tracks cover endings
after   "You ended three of four moves on cover."
```

N5, the machinery on screen. The generator reached inside the engine and put a scalar in front of the player.

```
before  "Your periodicity is past the cutoff. I counted it."
critic  N5: names the internal periodicity scalar and its threshold; the
        player is shown a read, not a debug value
after   "You looped a pattern."
```

H7 and H1, on the journey hazards. Two `Teaches` fields named mechanics that are not in the Act 3 vocabulary, and `train_leap` on day 3 ended with "Through the open door, later, the coast." The coast is day 5. The critic caught the anachronism and the repair deleted the sentence.

The remaining findings: two more N4s, two N6 register slips, two N3 lines citing an observable they never reference, two H4 pursuers that adapt, and four the critic filed under its own labels of `lore` and `tone` rather than a rule id, which is worth noting because those are not rules I gave it.

6. What the gate caught anyway

The critic is not the last line. `validators.py` is: 21 deterministic rules, re-run after the critic and the repair have both finished, and the only thing in the pipeline that can stop a row. On this run it stopped four, and the four divide into three different failure modes.

The critic was wrong, and the repair made the row worse. The critic asserted that `limp_onset` is not a canon beat id. It is one of the eight rows in the Chronicler beat table in GDD §4. The repair agent did as it was told and changed the field to `shelter_escape`, which genuinely does not exist.

```
generated  "CallbackBeat": "limp_onset"      <- correct
critic   A5  "No canon beat id named limp_onset exists"  <- false
repaired  "CallbackBeat": "shelter_escape"  <- now actually wrong
GATE     A5  CallbackBeat 'shelter_escape' is not a canon beat id
shipped   the authored fallback row
```

A critic that hallucinates a rule violation is worse than a critic that misses one, because the repair stage trusts it. The only reason that row did not reach Unreal is that `CANON_BEATS` is a list in a Python file and membership is not a matter of opinion.

A banned word survived both agents. `phase_train_yard` used the word "journey", which is on the bible's banned list in GDD §4. The critic looked at that row twice, filed two tone findings against other spans in it, and never noticed. A string match did.

Two rows the critic passed clean. `prior_attempt|clinical` and `gait_read|confident` both cite an observable in `ObservableCited` and then never reference it in the line, so the player cannot tell what the machine noticed. The critic passed both. Rule N3 checks the line against the lexicon for the field it claims, and failed both.

Final tally across 33 rows: **16 shipped as generated, 13 repaired, 4 replaced by the hand-authored fallback.** Every one of those four is a row where an agent was confidently wrong.

What running real output through the gate found in the gate

The first pass over un-coached generator output raised 29 gate failures, and most of them were my fault rather than the model's. Four fixes came out of it.

`find_adaptive` matched "adapts" as a substring, so a hazard whose `PursuerType` correctly read "never adapts" was flagged for adapting. It now checks the 24 characters before the verb for a word-boundary negation, and all six hazards stopped failing a rule they were obeying.

`FACT_DAY` dated "fence" to day 7 and "street" to day 9. Both fired on ordinary geography: a day-0 kennel yard has a fence, and the GDD's own day-0 beat is called `first_street`. What day 7 dates is the injury and what day 9 dates is the family's street, not the existence of wire or pavement. Both terms were removed.

The hazard prompt named rule H7 without listing the eight mechanics H7 accepts, so the generator wrote a sentence for `Teaches` on all six rows and the gate rejected all six. The nemesis and arena specs hand the model their closed values inside each key, which is why neither had the problem. `field_help` now states the vocabulary, and the same block types `CausesCondition` as a boolean, which the generator had been filling with the string "none".

A gate that over-fires is not a strict gate. Over-firing quietly routes good rows to the fallback table and hides the fact that generation is working.

7. Does it sound like Rex Machina

Mostly yes, and the failures are informative.

It sounds right where the retrieved chunk handed it a concrete noun. The hazard rows came out strongest, because the beat table rows are the most specific chunks in the corpus and the seed queries put the matching beat in front of every hazard. The read lines are the next strongest and they are the most constrained: nine words, a required observable, a banned register, no future tense. Under that much pressure there is little room to be generic.

It drifts in exactly one place, and the critic found the drift before I did. On `phase_train_yard` the generator wrote "Four covers, three exits, and a robot that pauses longer than it used to." The critic's note is better than anything I would have written about it: that is the agent's tactical readout, not what the dog sees, and the GDD puts the coarse tactical grid on the agent side. Same row, second finding: "the one bright rectangle on the board" names the grid abstraction inside a scene description. Both are the same failure, which is prose written from the schema's point of view instead of the animal's.

The `Light` and `SoundBed` fields are the weakest content in the set, and they are weakest because there was the least to retrieve. My GDD is an engineering document with almost no art direction in it. The weakness is a fair signal about the knowledge base rather than about the generator: the pipeline is only as specific as the canon it can retrieve. Filling that in is a job for the GDD.

One thing I did not expect. Four of the critic's 21 findings came back labelled `lore` and `tone` rather than with a rule id, and two of those four are the sharpest observations in the whole run. The rule list I gave it covers what can be checked mechanically. The critic spent part of its attention on what could not be, and that is where it earned its place.

8. Cost

Read the number below as a lower bound. Input is measured from the real assembled prompts. Output is projected from the response schemas, because a replay run has no live completion to count. The estimate also assumes one pass per stage and allows no retries. `estimate_cost.py` computes it rather than asserting it.

15 calls, 143,688 characters in, 36,856 out. At 3.5 to 4.0 characters per token that is 35.9k to 41.1k input tokens and 9.2k to 10.5k output. At Haiku 4.5 list pricing of \$1/M input and \$5/M output as of July 2026:

\$0.082 to \$0.094 for a full content build.

A build-time cost paid once by me, separate from the roughly \$0.036 per-playthrough runtime budget in GDD \$7. It counts only the prompts this pipeline builds.

9. Known limits

No live Haiku 4.5 call has ever been made from this project. The generator, critic and repair replies in `transcript/` were produced by subagents inside a Cowork session, each given one prompt file and nothing else. Every transcript file records `recorded_by: claude-opus-5 (subagent, given only the prompt file)`. Those agents could not read `validators.py`, `specs.py`, the fallback table or one another, so the defects, the catches, the false positive and the four gate failures are all real behaviour rather than authored illustrations. What is still missing is Haiku 4.5 specifically: a smaller model will make different and probably more mistakes. `python pipeline.py --live` runs the identical pipeline against it and records what comes back. See section 12.

Prose quality is one reader's opinion. I did not write the shipped rows, but I did choose the exemplars, the rules and the fallbacks the critic measures drift against, so my taste is baked into the standard even where it is not baked into the sentences.

The gate is only as good as its lexicons. Rules N3, N4, H1 and H4 all work off word lists in `specs.py`. Every one of them is a place where a defensible line could fail, or a bad one pass, on vocabulary rather than on meaning.

The retriever is lexical. A paraphrase that shares no vocabulary with the GDD will not retrieve. On a single-document corpus of identifiers I think that is the right trade, and §2 says why, but it is a limit and not a feature.

Generate 10, keep 3 is not implemented as a funnel. Class 5 teaches overgeneration followed by selection. This pipeline generates to spec and then filters through critic, repair and a deterministic gate. Same goal, different route, and stricter about what ships. The funnel is the better choice when the failure mode is blandness, because picking the best of ten raises the ceiling. The failure mode here was canon violation, which selection cannot fix and a gate can. It also spends fewer tokens.

Gold labels are mine. The retrieval numbers in §4 are measured against chunk sets I labelled myself, so they measure the retriever against my judgement of relevance and not against ground truth.

10. Files

pipeline.py	retrieve, generate, critique, repair, gate, write
kb.py	chunker and TF-IDF vector index
specs.py	the three content types, canon lexicons, seed queries
validators.py	the 21-rule deterministic gate
llm.py	live and replay providers
evaluate_retrieval.py	the measured numbers in section 4
estimate_cost.py	the cost figure in section 8
test_pipeline.py	168 assertions, no third-party runner
data/kb/gdd_rex_machina.md	the knowledge base: my GDD, verbatim
data/fallbacks.json	33 hand-authored rows
transcript/	15 real model turns, keyed by prompt hash
tools/build_fallbacks.py	regenerates data/fallbacks.json
tools/drive_agents.py	multi-pass driver that recorded transcript/
tools/build_readme_pdf.py	renders this file to README.pdf
out/DT_NemesisReads.json	24 rows + .csv + FNemesisReadRow.h
out/DT_JourneyHazards.json	6 rows + .csv + FJourneyHazardRow.h
out/DT_ArenaPhases.json	3 rows + .csv + FArenaPhaseRow.h
out/EVIDENCE.md	self-generating: gap, query, chunks, every critic finding with the repair that followed, every gate stop. Rebuilt on every run.
out/retrieval_traces.md	query, chunks and output side by side
out/retrieval_eval.md	the full evaluation, every query and every hit
out/run_report.json	every critic finding and every gate finding
architecture.mermaid	the diagram
README.pdf	this file, for reading rather than grepping

Importing the DataTables in Unreal needs UHT to run first. Regenerate project files and rebuild, and the Content Browser will offer the row types.

11. Consistency with Assignments #2 and #3

Assignment #3's README said this: "Assignment #4 replaces that selection with real vector retrieval; the packet contract on either side does not change." That is what happened. The Beat Foundry's Lorekeeper was handed all 24 bible entries and picked from them, because at roughly 600 tokens embedding cost more than it saved. This pipeline indexes the GDD and retrieves by cosine similarity, and the grounding packet that reaches the generator has the same shape it had before.

Nothing here contradicts the shipped A2 GDD. The pipeline reads the 15 rounds, the 3 phases at 33% and 66%, the positional approach meter, the 4-move window, `dir_freq_20`, `periodicity`, `read_category` suppression, the eight beat ids with their word budgets, and the three never-generated climax beats out of the same document. None of it is restated from memory. Three things this assignment settles that the GDD left open are marked as design decisions in §1 and §5 rather than presented as retrieved canon: the eight read categories, the register ladder for the three charge bands, and the six hazards with their day stamps.

12. Running it live

Everything above reproduces offline. To run the identical pipeline against Claude Haiku 4.5, on Windows:

```
cd "D:\Side Projects\AI Game Dev Course\Deliverables\Sritej Canchi - Assignment 4 - The Canon Index"
pip install anthropic
set ANTHROPIC_API_KEY=sk-ant-...
python pipeline.py --live
```

Fifteen calls. A few cents. Under a minute.

Nothing else changes: same knowledge base, same chunker, same retrieval, same prompts, same 21 gate rules, same authored fallbacks behind them.

Three things happen on a live run. `out/` is rewritten from what Haiku actually produced. `transcript/` is rewritten too, because `LiveProvider` records every exchange under the same prompt hash, so the recorded turns are replaced in place by real ones and the run reproduces offline afterwards. And `run_report.json` carries real token counts and a real cost instead of nulls.

The run is built to survive a bad reply rather than abort on one. A generator batch that comes back short, fenced or malformed gets one retry with the failure quoted back, then pads from the authored table. A critic or repair reply that cannot be parsed is skipped with a warning and the rows fall through to the gate. The floor never moves. No row reaches a `DataTable` without passing 21 deterministic checks, whatever the model does.

Sections 5, 6 and 7 of this README describe the run recorded here, and the defects Haiku makes will not be the defects in that run. You do not have to wait for someone to rewrite them. `out/EVIDENCE.md` is regenerated from scratch on every run and carries the same material in the same shape: the gap, the query, the retrieved chunks, every critic finding with the rule it cited and the repair that followed, field by field, and every row the gate stopped afterwards. Read that file first after a live run. `out/run_report.json` is the same data unformatted.

A live run overwrites `transcript/`. Keep a copy first if you want to be able to show both runs side by side.